

Java: Sub-rotinas (métodos)

Vetores e manipulação de vetores

Prof. Dionizio Porfírio
Unichristus • 2026.2

Objetivos

Ao final da aula, o você deverá ser capaz de:

- Compreender o conceito de sub-rotina em programação.
- Identificar os métodos como formas de implementar sub-rotinas em Java.
- Diferenciar métodos com retorno de métodos void.
- Declarar métodos utilizando a sintaxe adequada.
- Utilizar parâmetros para enviar informações aos métodos.
- Retornar valores por meio do comando return.
- Chamar métodos dentro de um programa.
- Dividir um programa em partes menores, organizadas e reutilizáveis.

Sub-rotinas em Java (métodos)

Java oferece suporte ao paradigma orientado a objetos e organiza seus programas por meio de classes. Dentro das classes, são definidos métodos, que correspondem a blocos de código responsáveis por executar tarefas específicas.

Um método pode receber informações para realizar seu processamento e também pode devolver um resultado. Dessa forma, os métodos permitem dividir um programa em partes menores, facilitando sua organização, compreensão e reutilização.

```
public static int somar(int numero1, int numero2) {  
    return numero1 + numero2;  
}
```

Nesse exemplo, o método somar recebe dois números inteiros, realiza a adição entre eles e devolve o resultado.

Estrutura de um método

Ao declarar um método, é necessário informar algumas características importantes: o modificador de acesso, a indicação de que ele é estático, o tipo do valor que será retornado, o nome do método e os parâmetros necessários para sua execução.

```
public static double calcularMedia(double nota1, double nota2) {  
    double media = (nota1 + nota2) / 2;  
    return media;  
}
```

A palavra `public` indica que o método pode ser acessado por outras classes. A palavra `static` permite que o método seja executado sem a criação de um objeto. O tipo `double` informa que o método devolverá um número real. Já `nota1` e `nota2` são os parâmetros utilizados no cálculo.

O comando `return` encerra a execução do método e devolve o resultado para o trecho do programa que realizou a chamada.

Chamando um método

Depois de declarado, um método pode ser utilizado por meio de uma chamada. Para isso, escreve-se o nome do método e, entre parênteses, são informados os valores correspondentes aos seus parâmetros.

```
public class Exemplo {  
  
    public static double calcularMedia(double nota1, double nota2) {  
        double media = (nota1 + nota2) / 2;  
        return media;  
    }  
  
    public static void main(String[] args) {  
        double resultado = calcularMedia(8.0, 6.0);  
  
        System.out.println("Média: " + resultado);  
    }  
}
```

Chamando um método

Nesse exemplo, o método main chama o método calcularMedia. Os valores 8.0 e 6.0 são enviados como argumentos, respectivamente, para os parâmetros nota1 e nota2. O valor retornado pelo método é armazenado na variável resultado.

A chamada de um método permite reaproveitar um conjunto de comandos sem precisar escrevê-lo novamente em diferentes partes do programa.

```
public class Exemplo {  
  
    public static double calcularMedia(double nota1, double nota2) {  
        double media = (nota1 + nota2) / 2;  
        return media;  
    }  
  
    public static void main(String[] args) {  
        double resultado = calcularMedia(8.0, 6.0);  
  
        System.out.println("Média: " + resultado);  
    }  
}
```

Parâmetros e argumentos

Os parâmetros são variáveis declaradas na definição do método. Eles indicam quais informações o método precisa receber para realizar sua tarefa.

```
public static int somar(int numero1, int numero2) {  
    return numero1 + numero2;  
}
```

Nesse método, numero1 e numero2 são parâmetros do tipo int. Eles receberão os valores utilizados durante a chamada do método.

Quando o método é chamado, os valores enviados são chamados de argumentos:

A quantidade, a ordem e os tipos dos argumentos devem ser compatíveis com os parâmetros definidos no método.

Retornos

Nem todo método precisa devolver um valor. Quando o método apenas executa uma ação, como apresentar uma mensagem ou exibir um resultado na tela, utiliza-se a palavra void no lugar do tipo de retorno.

```
public static void mostrarMensagem() {  
    System.out.println("Bem-vindo ao programa!");  
}
```

O método mostrarMensagem não recebe parâmetros e não retorna nenhum valor. Sua finalidade é apenas executar o comando responsável por exibir uma mensagem.

Para utilizar esse método, basta realizar sua chamada:

```
mostrarMensagem();
```

Como o método possui tipo de retorno void, não é necessário utilizar o comando return para devolver um resultado.

Método void com parâmetros

Um método do tipo void também pode receber parâmetros. Nesse caso, ele utiliza as informações recebidas para realizar uma ação, mas não devolve nenhum valor ao trecho que o chamou.

```
public static void mostrarDobro(int numero) {  
    System.out.println("O dobro é: " + (numero * 2));  
}
```

O método `mostrarDobro` recebe um número inteiro por meio do parâmetro `numero`. Em seguida, calcula o dobro desse valor e mostra o resultado na tela.

Observe que o método não possui o comando `return`, pois sua função é apenas apresentar uma informação.

```
mostrarDobro(7);
```

Variáveis locais

As variáveis declaradas dentro de um método são chamadas de variáveis locais. Elas podem ser utilizadas somente dentro do método em que foram declaradas.

```
public static double calcularMedia(double nota1, double nota2) {  
    double media = (nota1 + nota2) / 2;  
    return media;  
}
```

Nesse exemplo, a variável `media` é local ao método `calcularMedia`. Ela é criada quando o método começa a ser executado e só pode ser acessada dentro dele.

O método `main`, por exemplo, não pode utilizar diretamente a variável `media`. Para obter esse valor, é necessário chamar o método e armazenar o resultado retornado em outra variável.

Retorno de vários valores e alteração de arrays

Em Java, um método não pode retornar diretamente várias variáveis independentes, como uma função que devolve separadamente uma soma, uma média e um maior valor. Cada método possui apenas um retorno. Entretanto, esse retorno pode ser um array ou um objeto contendo vários resultados.

Quando os valores possuem o mesmo tipo e representam uma sequência, podemos retornar um array:

```
public static int[] calcular(int numero) {  
    int dobro = numero * 2;  
    int triplo = numero * 3;  
  
    return new int[] {dobro, triplo};  
}
```

Retorno de vários valores e alteração de arrays

A utilização do método pode ser feita da seguinte forma:

```
int[] resultados = calcular(5);  
  
System.out.println("Dobro: " + resultados[0]);  
System.out.println("Triplo: " + resultados[1]);
```

Nesse exemplo, o método retorna um único array, mas esse array contém dois valores.

Retorno de vários valores e alteração de arrays

Também é possível criar um método que altere diretamente os elementos de um array recebido como parâmetro. Como o array é um objeto, o método consegue modificar seus valores:

```
public static void dobrarValores(int[] numeros) {  
    for (int i = 0; i < numeros.length; i++) {  
        numeros[i] = numeros[i] * 2;  
    }  
}
```

O método pode ser chamado assim:

```
int[] valores = {2, 4, 6};  
  
dobrarValores(valores);  
  
for (int valor : valores) {  
    System.out.println(valor);  
}
```

O array não precisou ser retornado porque o método alterou diretamente seus elementos. A variável valores e o parâmetro numeros fazem referência ao mesmo array na memória.

Exercício

- 1.** Faça uma sub-rotina que receba um número inteiro e positivo N como parâmetro e retorne a soma dos números inteiros existentes entre o número 1 e N (inclusive).
- 2.** Crie uma sub-rotina que receba três números inteiros como parâmetros, representando horas, minutos e segundos, e os converta em segundos. Exemplo: 2h, 40min e 10s correspondem a 9.610 segundos.
- 5.** Faça uma sub-rotina que receba um valor inteiro e verifique se ele é positivo ou negativo.
- 6.** Crie uma sub-rotina que receba como parâmetro a altura (alt) e o sexo de uma pessoa e retorne seu peso ideal. Para homens, deverá calcular o peso ideal usando a fórmula: $\text{peso ideal} = 72.7 * \text{alt} - 58$; para mulheres: $\text{peso ideal} = 62.1 * \text{alt} - 44.7$.
- 7.** Elabore uma sub-rotina que leia um número não determinado de valores positivos e retorne a média aritmética desses valores. Terminar a entrada de dados com o valor zero.
- 8.** Faça uma sub-rotina que receba um valor inteiro e positivo, calcule e mostre seu fatorial.
- 9.** Crie uma sub-rotina que receba como parâmetro um valor inteiro e positivo e retorne a soma dos divisores desse valor.

Exercício

- 13.** Foi realizada uma pesquisa entre 15 habitantes de uma região. Os dados coletados de cada habitante foram: idade, sexo, salário e número de filhos.

Faça uma sub-rotina que leia esses dados armazenando-os em vetores. Depois, crie sub-rotinas que recebam esses vetores como parâmetro e retornem a média de salário entre os habitantes, a menor e a maior idade do grupo e a quantidade de mulheres com três filhos que recebem até R\$ 500,00 (utilize uma sub-rotina para cada cálculo).

- 23.** Crie um programa que receba três valores (obrigatoriamente maiores que zero), representando as medidas dos três lados de um triângulo.

Elabore sub-rotinas para:

- determinar se esses lados formam um triângulo (sabe-se que, para ser triângulo, a medida de um lado qualquer deve ser inferior ou igual à soma das medidas dos outros dois).
- determinar e mostrar o tipo de triângulo (equilátero, isósceles ou escaleno), caso as medidas formem um triângulo.

Todas as mensagens deverão ser mostradas no programa principal.

- 25.** Crie um programa que receba o número dos 10 alunos de uma sala, armazenando-os em um vetor, junto com as notas obtidas ao longo do semestre (foram realizadas quatro avaliações). Elabore sub-rotinas para:

- determinar e mostrar a média aritmética de todos os alunos;
- indicar os números dos alunos que deverão fazer recuperação, ou seja, aqueles com média inferior a 6.

Obrigado!